

Assignment 2:

Advanced Password Security Assessment Using Hashcat, John the Ripper, and AI Techniques

Nouf Abdullah Alquhiadan 445008277

Q1: Hashing Weakness Analysis

Explain why fast hashing algorithms (MD5, SHA1, SHA256) are insecure for password storage.

Required:

- Speed vs security

Fast hashing algorithms such as MD5, SHA1, and SHA256 are insecure for password storage because they are designed to compute hashes very quickly. This speed is useful for file integrity checks, but it is dangerous for passwords because attackers can test millions or billions of guesses per second using GPUs. A secure password hashing method should be intentionally slow, so each password guess costs more time and computing power.

.....

- Collision concept

A collision happens when two different inputs produce the same hash output. In weak algorithms like MD5 and SHA1, collisions have been demonstrated in practice. For password storage, the bigger issue is usually brute-force speed, but collisions still show that the algorithm is not cryptographically strong and should not be trusted for modern security use.

.....

- Real-world Impact:

If a database stores passwords using MD5, SHA1, or even plain SHA256 without protection such as salting and key stretching, attackers who steal the hash file can crack many passwords quickly. Common passwords like “password” or “123456” are often recovered in seconds using dictionary attacks and tools such as Hashcat or John the Ripper. This can lead to account compromise, credential reuse attacks, and wider data breaches.

Q2: Salting + Key Stretching

Explain:

- **Salting:**

Salting means adding a unique random value to each password before hashing it. This ensures that even if two users have the same password, their final hashes will be different.

Example:

Password: Ahmed123

Salt: xyz

Final input: Ahmed123xyz

A salt prevents attackers from using precomputed rainbow tables and makes bulk cracking less efficient because each password must be attacked separately.

- **Key stretching (bcrypt / Argon2)**

Key stretching means hashing the password in a way that is deliberately slow and computationally expensive. Algorithms such as bcrypt and Argon2 are designed for password storage, unlike MD5 or SHA1. They increase the time and resources needed for every password guess.

- Why they defeat brute-force attacks

Salting defeats precomputed attacks and prevents identical passwords from producing identical hashes. Key stretching defeats brute-force attacks by making every guess slow and expensive. Together, they greatly reduce the effectiveness of dictionary, brute-force, and GPU-based cracking attacks.

Q3: Password Entropy Calculation

Note : Entropy measures password unpredictability.

Estimate entropy for:

Ahmed123

Welcome2024

T9\$kP!2vL@7q

Ahmed123

This password uses uppercase letters, lowercase letters, and digits. A reasonable character set estimate is about 62 characters. Length is 8.

Entropy $\approx 8 \times \log_2(62) \approx 47.6$ bits

Welcome2024

This password also uses uppercase letters, lowercase letters, and digits. Character set estimate is about 62. Length is 11.

Entropy $\approx 11 \times \log_2(62) \approx 65.5$ bits

T9\$kP!2vL@7q

This password uses uppercase letters, lowercase letters, digits, and symbols. A reasonable character set estimate is about 94 characters. Length is 12.

Entropy $\approx 12 \times \log_2(94) \approx 78.7$ bits

Explain which is strongest and why?

The strongest password is T9\$kP!2vL@7q because it is longer and uses a wider range of character types, including symbols. This gives it the highest entropy and makes it harder to guess with dictionary, rule-based, and brute-force attacks.

.....

PART 2: EXTENDED PRACTICAL LAB (CORE OF ASSIGNMENT)

Task 1: Create Controlled Password Dataset

Create 10 passwords with increasing complexity:

Level	Description	Example
Weak	common word	password
Medium	word + numbers	Ahmed123
Strong	mixed symbols	A@9kL!2x
Very Strong	Random	T9\$kP!2vL@7q
.....		

Write your dataset:

password
123456
Ahmed123
Welcome2024
Omar2024
B@7mK13y
Netw0rk55
R7Vq1WxQ2k
Kz\$4n!PMw80
M03xL17N#9q

The dataset starts with weak common passwords and gradually moves to more complex passwords with mixed uppercase letters, lowercase letters, digits, and symbols.

Level	Description	Example
Weak	Common word	password
Weak	Common number sequence	123456
Medium	Word + numbers	Ahmed123
Medium	Word + year	Welcome2024
Medium	Name + year	Omar2024
Strong	Mixed letters + symbol + numbers	B@7mK13y
Strong	Mixed letters + numbers	Netw0rk55

Very Strong	Random mixed letters + numbers	R7Vq1WxQ2k
Very Strong	Random mixed letters + symbols + numbers	Kz\$4n!PMw80
Very Strong	Random mixed letters + symbols + numbers	M03xL17N#9q

```
GNU nano 8.2 passwords.txt *
password
123456
Ahmed123
Welcome2024
Omar2024
B@7mK13y
Netw0rk55
R7Vq1WxQ2k
Kz$4n!PMw80
M03xL17N#9q
```

Task 2: Generate Multiple Hash Types

Note: Check your hash:

- MD5 → -m 0
- SHA1 → -m 100

- SHA256 → -m 1400

Generate hashes using 3 algorithms:

`echo -n "password" | md5sum`

`echo -n "password" | sha1sum`

`echo -n "password" | sha256sum`

```

pnu-kali@kali: ~
File Actions Edit View Help

(pnu-kali@kali)~$ echo -n "password" | md5sum
5f4dcc3b5aa765d61d8327deb882cf99 -

(pnu-kali@kali)~$ echo -n "123456" | md5sum
e10adc3949ba59abbe56e057f20f883e -

(pnu-kali@kali)~$ echo -n "Ahmed123" | md5sum
ed36c2b690f236a4529db4d406707839 -

(pnu-kali@kali)~$ echo -n "Welcome2024" | md5sum
94b7384f4029d0e69461206622304a26 -

(pnu-kali@kali)~$ echo -n "Omar2024" | md5sum
55ac7984f082574abed80ba76811c69 -

(pnu-kali@kali)~$ echo -n "BQ7mK13y" | md5sum
a3fb082e8a73c9e7389d732d58452511 -

(pnu-kali@kali)~$ echo -n "NetW0rk55" | md5sum
6a0b751ade52ca249b4197b64e67d106 -

(pnu-kali@kali)~$

(pnu-kali@kali)~$

(pnu-kali@kali)~$ echo -n "R7Vq1WxQ2k" | md5sum
6fa8585952c6f9568b61d8fd365a5557 -

(pnu-kali@kali)~$ echo -n "Kz$An1PMw80" | md5sum
9c27fb30a66b37d850eb83dc523ec417 -

(pnu-kali@kali)~$ echo -n "M03xL17rN9q" | md5sum
fc3468ece3824a1723813369995f4186 -

```

```

(nouf@kali2)~$ echo -n "password" | sha1sum
echo -n "123456" | sha1sum
echo -n "Ahmed123" | sha1sum
echo -n "Welcome2024" | sha1sum
echo -n "Omar2024" | sha1sum
echo -n "BQ7mK13y" | sha1sum
echo -n "NetW0rk55" | sha1sum
echo -n "R7Vq1WxQ2k" | sha1sum
echo -n "Kz$An1PMw80" | sha1sum
echo -n "M03xL17rN9q" | sha1sum
5baa61e4c9b03f3f0682250b6cf8331b7ee68fd8 -
7c4a8d09ca3762af61e59520943dc26494f8941b -
174b681482541048ba75e942a21a0cab36377c8e -
b74df8452be95e3bcf8744ccf8c237bc2915f7ab -
2100394f9823958e0ed5b2c81e24095f91fc86 -
c36d47c14ceded5cadd77040e4d168587f485dd1 -
e4d2b48ace7ad8d69318fc203d1b97a72088f207 -
57362aa5e2a6836e58ae5443794e39f6fe5152e -
a16d1a4efafe86c80e906cc1aa74acd7e3af9ef1 -
fae83db89c4e7770bff719e84d028b3c53d2c983 -

(nouf@kali2)~$ echo -n "password" | sha256sum
echo -n "123456" | sha256sum
echo -n "Ahmed123" | sha256sum
echo -n "Welcome2024" | sha256sum
echo -n "Omar2024" | sha256sum
echo -n "BQ7mK13y" | sha256sum
echo -n "NetW0rk55" | sha256sum
echo -n "R7Vq1WxQ2k" | sha256sum
echo -n "Kz$An1PMw80" | sha256sum
echo -n "M03xL17rN9q" | sha256sum
5e88498da28047f51d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8 -
80990ee6fecdc1c293a3e29280e686cfbc1f5d5a8aff3ca12020c923ad6cc92 -
df8e7c32787fb73d95121c102928b743cab9296ba7fb77f3a368cf8995c06 -
7f24cc5d7da8e20a857b903df2b2b06556266abfd85abd4ce77198afa176f06 -
5575848b016b80b648171b7deb389f501a7a2aa6b84e284ce1416df3e7bdfa69 -
9644cf04df3ded6e3b97d4d4e65055b65fae312f626e4b438a1c91c911ee9a8a -
9079f700d3b80fd7ab7288eb9385b14cf1ba37d302a8e0c47c02a90d65d6181b -
dd80b310934a10a0eac7fd117f00953899e7dbcaa6809459f5fb7643d3cc8d5d -
8665b0c23100f5e01d27e41f07219d0c8d6f3906ff90fd2292e3bf6b0b5fa8f1f -
1240c7f1a0812183f2d3f9c2929808af45be5b54cda8d9c305d9143818ba7f2 -

```

Save into:

hashes_md5.txt

hashes_sha1.txt

hashes_sha256.txt

```
(pnu-kali@kali)-[~]
$ echo -n 'password' | sha256sum | awk '{print $1}' >> hashes_sha256.txt
echo -n '123456' | sha256sum | awk '{print $1}' >> hashes_sha256.txt
echo -n 'Ahmed123' | sha256sum | awk '{print $1}' >> hashes_sha256.txt
echo -n 'Welcome2024' | sha256sum | awk '{print $1}' >> hashes_sha256.txt
echo -n 'Omar@2024' | sha256sum | awk '{print $1}' >> hashes_sha256.txt
echo -n 'B@7mK!3y' | sha256sum | awk '{print $1}' >> hashes_sha256.txt
echo -n 'Net@work55' | sha256sum | awk '{print $1}' >> hashes_sha256.txt
echo -n 'R7#vQ!9wX@2k' | sha256sum | awk '{print $1}' >> hashes_sha256.txt
echo -n 'Kz$4nP!mW8@j' | sha256sum | awk '{print $1}' >> hashes_sha256.txt
echo -n 'M@3xL!7rN#9q' | sha256sum | awk '{print $1}' >> hashes_sha256.txt
cat hashes_sha256.txt
5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92
df8e7c6127e87fb73d95121c1d2928b743cab6296ba2fb77ff3a368c8f895c06
7f24cc5d7da8e20a857b903df2b2b06556266abfd85abdc4ce77108afa176f06
d925c5667043cb9b2879e160ad4cb065977b9810a6f174aaa5db532898a8d4fc
17767113c495a3b64128fa1ed0f6f06ddd9bfa4191d426b56ef3013758a52b81
7e90a36be9f0ee3d4411a23aeff4f6ca3377c4fec34963de63271b3e4b1e3680
e5edbcb1b4d78014977f7be88b8a5536e328a25be266c21cea19f0ddcdadafc1c24
dae91fecfd76f7de06b183fa0cd947a1b9bce4e11750b23f7f3b2c71cf783073
350109b6d50810b53152982d18b49264e6ed7e424f1dd104566c31c7940ee216
```

```
(pnu-kali@kali)-[~]
$ echo -n 'password' | sha1sum | awk '{print $1}' >> hashes_sha1.txt
echo -n '123456' | sha1sum | awk '{print $1}' >> hashes_sha1.txt
echo -n 'Ahmed123' | sha1sum | awk '{print $1}' >> hashes_sha1.txt
echo -n 'Welcome2024' | sha1sum | awk '{print $1}' >> hashes_sha1.txt
echo -n 'Omar@2024' | sha1sum | awk '{print $1}' >> hashes_sha1.txt
echo -n 'B@7mK!3y' | sha1sum | awk '{print $1}' >> hashes_sha1.txt
echo -n 'Net@work55' | sha1sum | awk '{print $1}' >> hashes_sha1.txt
echo -n 'R7#vQ!9wX@2k' | sha1sum | awk '{print $1}' >> hashes_sha1.txt
echo -n 'Kz$4nP!mW8@j' | sha1sum | awk '{print $1}' >> hashes_sha1.txt
echo -n 'M@3xL!7rN#9q' | sha1sum | awk '{print $1}' >> hashes_sha1.txt
cat hashes_sha1.txt
5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
7c4a8d09ca3762af61e59520943dc26494f8941b
174b681482541048ba75e942a21a0cab3637fc8e
b74df8452be95e3bcf8744ccf8c237bc2915f7ab
e5e13785ca206e44b9fdb484a136a47bb8f54dd3
d540c2bbade4c6b9158556bb1b11fb61fac64f8a
71ceb22415c2df3cb853944d87d24812d6a638b3
c5fa271b26995f15825d2debe30842e5c137ad94
e74fca92eba63825192aede8f0177c262cfd06e6
2feccf7b6a5c600e5240d37cd0acaae12326ac27
```

```
(pnu-kali@kali)-[~]
```

.....

.....

```

(nouf@kali2)-[~]
$ echo -n 'password' | md5sum | awk '{print $1}' >> hashes_md5.txt
echo -n '123456' | md5sum | awk '{print $1}' >> hashes_md5.txt
echo -n 'Ahmed123' | md5sum | awk '{print $1}' >> hashes_md5.txt
echo -n 'Welcome2024' | md5sum | awk '{print $1}' >> hashes_md5.txt
echo -n 'Omar2024' | md5sum | awk '{print $1}' >> hashes_md5.txt
echo -n 'B@7mK13y' | md5sum | awk '{print $1}' >> hashes_md5.txt
echo -n 'Netw0rk55' | md5sum | awk '{print $1}' >> hashes_md5.txt
echo -n 'R7Vq1WxQ2k' | md5sum | awk '{print $1}' >> hashes_md5.txt
echo -n 'Kz$4n!PMw80' | md5sum | awk '{print $1}' >> hashes_md5.txt
echo -n 'M03xL17N#9q' | md5sum | awk '{print $1}' >> hashes_md5.txt

(nouf@kali2)-[~]
$ cat hashes_md5.txt
5f4dcc3b5aa765d61d8327deb882cf99
e10adc3949ba59abbe56e057f20f883e
ed36c2b690f236a4529db4d406707039
94b7384f4029d0e69461206622304a26
a8ee476ab4c91df1dd283291d4964a10
61056491bc265e32d3adde29c20268c6
254a591130920403ec58db8ef1a2e884
332fdf82f2dd8f65ec6975591e47be24
3dd6fbf6d0acbdca0e2e9a5d6e2ad689
62d7466aaad8f28787fccf931de4ee23

```

Question:

- Compare hash lengths and structure
- MD5 = 32 hex characters = 128 bits
- SHA1 = 40 hex characters = 160 bits
- SHA256 = 64 hex characters = 256 bits
- All three hashes are written in hexadecimal format, but they differ in output length and security level. MD5 is the shortest and weakest. SHA1 is longer than MD5 but still considered insecure. SHA256 is the longest and strongest among the three, but it is still too fast to be safe for password storage when used alone.

Task 3: Hash Identification Challenge

Without looking at commands:

- Identify hash type manually
- Then verify using AI

Explain method:

```
(pnu-kali@kali)~$ cat hashes_sha1.txt
5baa61e4c9b93f3f0682250b6cf8331b7ee68fd8
7c4a8d09ca3762af61e59520943dc26494f8941b
174b681482541048ba75e942a21a0cab3637fc8e
b74df8452be95e3bcf8744ccf8c237bc2915f7ab
e5e13785ca206e44b9fdb484a136a47bb8f54dd3
d540c2bbade4c6b9158556bb1b11fb61fac64f8a
71ceb22415c2df3cb853944d87d24812d6a638b3
c5fa271b26995f15825d2debe30842e5c137ad94
e74fca92eba63825192aede8f0177c262cfd06e6
2fecf7b6a5c600e5240d37cd0acaee12326ac27

(pnu-kali@kali)~$ cat hashes_sha256.txt
5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8
8d969eef6ecad3c29a3a629280e686cf0c3f5d5a86aff3ca12020c923adc6c92
df8e7c6127e87fb73d95121c1d2928b743cab6296ba2fb77ff3a368c8f895c06
7f24cc5d7da8e20a857b903df2b2b06556266abfd85abdc4ce77108afa176f06
d925c5667043cb9b2879e160ad4cb065977b9810a6f174aaa5db532898a8d4fc
17767113c495a3b64128fa1ed0f6f06dd9bfa4191d426b56ef3013758a52b81
7e90a36be9f0ee3d4411a23aef4f6ca3377c4fec34963de63271b3e4be3680
e5edbc1b4d78014977f8e88b8a5536e328a25be266c21cea19f0ddccddafc1c24
dae91fecfd767de06b183fa0cd947a1b9bce4e11750b23f7f3b2c71cf783073
350109b6d50810b53152982d18b49264e6ed7e424f1dd104566c31c7940ee216

(pnu-kali@kali)~$ cat hashes_md5.txt
5f4dcc3b5aa765d61d8327deb882cf99
e10adc3949ba59abbe56e057f20f883e
ed36c2b690f236a4529db4d406707039
94b7384f4029d0e69461206622304a26
55ac7984f082574abedb80ba76811c69
a3fb082e8a73c9e7389d732d58452511
6a0b751ade52ca249b4197b64e67d106
6fa8585952c6f9568b61d8fd365a5557
9c27fb30a66b37d850eb83dc523ec417
fc3468ece3824a1723813369995f4186
```

The hash type can be identified manually by checking its length and format.

- 32 hexadecimal characters usually indicate **MD5**
- 40 hexadecimal characters usually indicate **SHA1**
- 64 hexadecimal characters usually indicate **SHA256**

I first counted the number of hexadecimal characters in the hash. Based on the length, I identified the type manually. Then I verified the result using AI by entering the hash into ChatGPT, and the result matched my manual identification.

```
Use the hash length to identify the type:
• 32 hex characters → MD5
• 40 hex characters → SHA1
• 64 hex characters → SHA256

Results
• hashes_md5.txt → MD5
• hashes_sha1.txt → SHA1
• hashes_sha256.txt → SHA256
```

Task 4: Multi-Attack Strategy Using Hashcat :

Attack 1: Dictionary Attack

`sudo hashcat -m 0 -a 0 -o cracked.txt hashes_md5.txt`

`/usr/share/wordlists/rockyou.txt`

```
(pnu-kali@kali)-[~]
└─$ sudo gunzip /usr/share/wordlists/rockyou.txt.gz
(pnu-kali@kali)-[~]
└─$ sudo hashcat -m 0 -a 0 -o cracked.txt hashes_md5.txt /usr/share/wordlists/rockyou.txt
hashcat (v6.1.1) starting...

OpenCL API (OpenCL 2.0 pocl 1.8 Linux, None+Asserts, RELOC, LLVM 9.0.1, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #1: pthread-12th Gen Intel(R) Core(TM) i9-12900, 8784/8848 MB (4096 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 10 digests; 10 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0=0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Applicable optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Using pure kernels enables cracking longer passwords but for the price of drastically reduced performance.
If you want to switch to optimized backend kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.

Host memory required for this attack: 65 MB

Dictionary cache built:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344392
* Bytes.....: 139921587
* Keyspace...: 14344385
* Runtime... : 0 secs
```

```
* Runtime ... : 0 secs

Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Name.....: MD5
Hash.Target.....: hashes_md5.txt
Time.Started.....: Thu Apr  9 05:49:51 2026 (2 secs)
Time.Estimated...: Thu Apr  9 05:49:53 2026 (0 secs)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 9028.3 kH/s (0.14ms) @ Accel:1024 Loops:1 Thr:1 Vec:8
Recovered.....: 3/10 (30.00%) Digests
Progress.....: 14344385/14344385 (100.00%)
Rejected.....: 0/14344385 (0.00%)
Restore.Point...: 14344385/14344385 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-1 Iteration:0-1
Candidates.#1...: $HEX[206b72697374656e616e6e65] → $HEX[042a0337c2a156616d6f732103]

Started: Thu Apr  9 05:49:32 2026
Stopped: Thu Apr  9 05:49:54 2026

(pnu-kali@kali)-[~]
└─$ sudo cat cracked.txt
e10adc3949ba59abbe56e057f20f883e:123456
5f4dcc3b5aa765d61d8327deb882cf99:password
ed36c2b690f236a4529db4d406707039:Ahmed123
```

.....

.....

.....

Record:

- Cracked passwords

password

123456

Ahmed123

.....

- Time taken

Seconds only

.....

The dictionary attack was successful because these passwords or close variants exist in common wordlists such as rockyou.txt. Hashcat compares each candidate word in the wordlist to the target hashes until a match is found.

Attack 2: Rule-Based Attack

```
sudo hashcat -m 0 -a 0 -o ruled.txt hashes_md5.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best66.rule --show
```

```
(pnu-kali@kali)-[~]
$ sudo hashcat -m 0 -a 0 -o ruled.txt hashes_md5.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule
hashcat (v6.1.1) starting...

OpenCL API (OpenCL 2.0 pocl 1.8 Linux, None+Asserts, RELOC, LLVM 9.0.1, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #1: pthread-12th Gen Intel(R) Core(TM) i9-12900, 8784/8848 MB (4096 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 10 digests; 10 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 77

Applicable optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Using pure kernels enables cracking longer passwords but for the price of drastically reduced performance.
If you want to switch to optimized backend kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.

INFO: Removed 3 hashes found in potfile.

Host memory required for this attack: 65 MB

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 1104517645
```

```
ATTENTION! Pure (unoptimized) backend kernels selected.
Using pure kernels enables cracking longer passwords but for the price of drastically reduced performance.
If you want to switch to optimized backend kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.

INFO: Removed 3 hashes found in potfile.

Host memory required for this attack: 65 MB

Dictionary cache hit:
* Filename..: /usr/share/wordlists/rockyou.txt
* Passwords.: 14344385
* Bytes.....: 139921507
* Keyspace..: 1104517645

Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Name.....: MD5
Hash.Target....: hashes_md5.txt
Time.Started...: Thu Apr  9 05:51:41 2026 (26 secs)
Time.Estimated.: Thu Apr  9 05:52:07 2026 (0 secs)
Guess.Base.....: File (/usr/share/wordlists/rockyou.txt)
Guess.Mod.....: Rules (/usr/share/hashcat/rules/best64.rule)
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 41861.9 KH/s (7.09ms) @ Accel:1024 Loops:77 Thr:1 Vec:8
Recovered.....: 3/10 (30.00%) Digests
Progress.....: 1104517645/1104517645 (100.00%)
Rejected.....: 0/1104517645 (0.00%)
Restore.Point...: 14344385/14344385 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-77 Iteration:0-77
Candidates.#1...: $HEX[206b72697374656e616e6e65] -> $HEX[04a156616d6f]

Started: Thu Apr  9 05:51:40 2026
Stopped: Thu Apr  9 05:52:00 2026
```

```
(pnu-kali@kali)-[~]
$ sudo hashcat -m 0 -a 0 hashes_md5.txt /usr/share/wordlists/rockyou.txt -r /usr/share/hashcat/rules/best64.rule --show
5f4dcc3b5aa765d61d8327deb882cf99:password
e10adc3949ba59abbe56e057f20f883e:123456
ed36c2b690f236a4529db4d406707039:Ahmed123

(pnu-kali@kali)-[~]
$
```

Explanation

The rule-based attack applies transformation rules to words from the dictionary. These rules can capitalize letters, append numbers, replace characters with symbols, and generate common password variations.

Explain:

- What transformation rules did ?

The transformation rules modified base dictionary words into realistic password variants, such as adding years, changing case, or substituting letters with symbols. This helps detect passwords that are not exact dictionary words but are still predictable.

.....

Result

The rule-based attack was more powerful than a plain dictionary attack because it tested many modified versions of common words. In this case, it successfully cracked passwords such as password, 123456, and Ahmed123.

.....

Attack 3: Mask Attack

Try different patterns:

?l?l?l?d?d

?u?l?l?d?d

?u?l?l?l?l?d?d

Note : `sudo hashcat -m 0 -a 3 -o h.txt hashes_md5 ?u?l?l?l?l?d?d -show`

```
(pnu-kali@kali)-[~]
$ sudo hashcat -m 0 -a 3 -o mask1.txt hashes_md5.txt ?l?l?l?l?d
hashcat (v6.1.1) starting...

OpenCL API (OpenCL 2.0 pocl 1.8 Linux, None+Asserts, RELOC, LLVM 9.0.1, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #1: pthread-12th Gen Intel(R) Core(TM) i9-12900, 8784/8848 MB (4096 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 10 digests; 10 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates

Applicable optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Brute-Force
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Using pure kernels enables cracking longer passwords but for the price of drastically reduced performance.
If you want to switch to optimized backend kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.

INFO: Removed 3 hashes found in potfile.

Host memory required for this attack: 65 MB

Approaching final keypace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Name.....: MD5
Hash.Target.....: hashes_md5.txt
Time.Started....: Thu Apr  9 06:12:09 2026 (0 secs)
Time.Estimated...: Thu Apr  9 06:12:09 2026 (0 secs)
Guess.Mask.....: ?l?l?l?l?d [6]
Guess.Queue.....: 1/1 (100.00%)
Speed.#1.....: 146.1 MH/s (6.20ms) @ Accel:1024 Loops:338 Thr:1 Vec:8
```

```
(nouf@kali2)-[~]
$ cat mask1.txt
cat: mask1.txt: No such file or directory
```

```
pnu-kali@kali: ~  
File Actions Edit View Help  
Recovered.....: 3/10 (30.00%) Digests  
pnu-kali@kali: ~  
File Actions Edit View Help  
Approaching final keyspace - workload adjusted.  
  
Session.....: hashcat  
Status.....: Exhausted  
Hash.Name.....: MD5  
Hash.Target.....: hashes_md5.txt  
Time.Started.....: Thu Apr 9 06:12:16 2026 (1 sec)  
Time.Estimated...: Thu Apr 9 06:12:17 2026 (0 secs)  
Guess.Mask.....: ?u?l?l?l?d?d [6]  
Guess.Queue.....: 1/1 (100.00%)  
Speed.#1.....: 195.6 MH/s (1.36ms) @ Accel:1024 Loops:84 Thr:1 Vec:8  
Recovered.....: 3/10 (30.00%) Digests  
Progress.....: 45697600/45697600 (100.00%)  
Rejected.....: 0/45697600 (0.00%)  
Restore.Point....: 67600/67600 (100.00%)  
Restore.Sub.#1...: Salt:0 Amplifier:672-676 Iteration:0-84  
Candidates.#1....: Zfyx83 → Xqqf59  
  
Started: Thu Apr 9 06:12:16 2026  
Stopped: Thu Apr 9 06:12:19 2026  
  
(pnu-kali@kali)-[~]  
$ sudo hashcat -m 0 -a 3 -o mask3.txt hashes_md5.txt ?u?l?l?l?d?d  
hashcat (v6.1.1) starting ...  
  
OpenCL API (OpenCL 2.0 pocl 1.8 Linux, None+Asserts, RELOC, LLVM 9.0.1, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]  
  
* Device #1: pthread-12th Gen Intel(R) Core(TM) i9-12900, 8784/8848 MB (4096 MB allocatable), 4MCU  
  
Minimum password length supported by kernel: 0  
Maximum password length supported by kernel: 256  
  
Hashes: 10 digests; 10 unique digests, 1 unique salts  
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates  
  
Applicable optimizers applied:  
* Zero-Byte  
* Early-Skip  
* Not-Salted  
* Not-Iterated  
* Single-Salt  
* Brute-Force  
* Raw-Hash  
  
ATTENTION! Pure (unoptimized) backend kernels selected.  
Using pure kernels enables cracking longer passwords but for the price of drastically reduced performance.  
If you want to switch to optimized backend kernels. append -O to your commandline.  
  
(nouf@kali2)-[~]  
$ cat mask2.txt  
cat: mask2.txt: No such file or directory  
  
(nouf@kali2)-[~]  
$ cat mask3.txt  
cat: mask3.txt: No such file or directory
```

Record:

- Which mask worked best:
- None of the tested masks worked
- Why :
- The tested masks did not match the structure of the passwords in the dataset. Some passwords were simple dictionary words, while others contained mixed letters, numbers, and symbols in patterns different from the tested masks. A mask attack is only effective when the attacker knows or correctly guesses the password structure.

Attack 4: Hybrid Attack (Most Important)

```
sudo hashcat -m 0 -a 6 -o h.txt hashes_md5.txt /usr/share/wordlists/rockyou.txt  
?u?!?!?!?!?d?d -show
```

Explain:

```
(pnu-kali@kali)-[~]  
$ sudo hashcat -m 0 -a 6 -o hybrid.txt hashes_md5.txt /usr/share/wordlists/rockyou.txt ?d?d?d?d  
hashcat (v6.1.1) starting ...  
  
OpenCL API (OpenCL 2.0 pocl 1.8 Linux, None+Asserts, RELOC, LLVM 9.0.1, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]  
  
* Device #1: pthread-12th Gen Intel(R) Core(TM) i9-12900, 8784/8848 MB (4096 MB allocatable), 4MCU  
  
Minimum password length supported by kernel: 0  
Maximum password length supported by kernel: 256  
  
Hashes: 10 digests; 10 unique digests, 1 unique salts  
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates  
  
Applicable optimizers applied:  
* Zero-Byte  
* Early-Skip  
* Not-Salted  
* Not-Iterated  
* Single-Salt  
* Raw-Hash  
  
ATTENTION! Pure (unoptimized) backend kernels selected.  
Using pure kernels enables cracking longer passwords but for the price of drastically reduced performance.  
If you want to switch to optimized backend kernels, append -O to your commandline.  
See the above message to find out about the exact limits.  
  
Watchdog: Hardware monitoring interface not found on your system.  
Watchdog: Temperature abort trigger disabled.  
  
INFO: Removed 3 hashes found in potfile.  
  
Host memory required for this attack: 65 MB  
  
Dictionary cache hit:  
* Filename ..: /usr/share/wordlists/rockyou.txt  
* Passwords..: 14344385  
* Bytes.....: 139921507  
* Keyspace...: 143443850000  
  
[s]tatus [p]ause [b]ypass [c]heckpoint [q]uit => █
```

- Why hybrid attack is powerful

A hybrid attack is powerful because it combines a dictionary wordlist with a mask pattern. This means it can test realistic password structures such as a common word followed by numbers or symbols. Many users create passwords in exactly this style, such as Welcome2024 or Ahmed123. Because of this, hybrid attacks are often more effective than a pure dictionary attack or a pure brute-force attack.

However, in this experiment the dictionary attack and the rule-based attack showed clearer successful results, while the hybrid attack did not produce stronger recovery in the recorded output

Attack 5: Incremental Brute Force (Advanced)

```
(pnu-kali@kali)-[~]
$ sudo hashcat -m 0 -a 3 hashes_md5.txt --increment --increment-min=1 --increment-max=4
hashcat (v6.1.1) starting ...

OpenCL API (OpenCL 2.0 pocl 1.8 Linux, None+Asserts, RELOC, LLVM 9.0.1, SLEEP, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #1: pthread-12th Gen Intel(R) Core(TM) i9-12900, 8784/8848 MB (4096 MB allocatable), 4MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 10 digests: 10 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates

Applicable optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Brute-Force
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Using pure kernels enables cracking longer passwords but for the price of drastically reduced performance.
If you want to switch to optimized backend kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Hardware monitoring interface not found on your system.
Watchdog: Temperature abort trigger disabled.

INFO: Removed 4 hashes found in potfile.

Host memory required for this attack: 65 MB

The wordlist or mask that you are using is too small.
This means that hashcat cannot use the full parallel power of your device(s).
Unless you supply more work, your cracking speed will drop.
For tips on supplying more work, see: https://hashcat.net/faq/morework

Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Name.....: MD5
Hash.Target.....: hashes_md5.txt
Time.Started.....: Thu Apr  9 06:16:40 2026 (0 secs)
```

```

Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Name.....: MD5
Hash.Target.....: hashes_md5.txt
Time.Started.....: Thu Apr  9 06:16:40 2026 (0 secs)
Time.Estimated...: Thu Apr  9 06:16:40 2026 (0 secs)
Guess.Mask.....: ?1 [1]
Guess.Charset....: -1 ?l?d?u, -2 ?l?d, -3 ?l?d*!$@_, -4 Undefined
Guess.Queue.....: 1/4 (25.00%)
Speed.#1.....: 71313 H/s (0.00ms) @ Accel:1024 Loops:62 Thr:1 Vec:8
Recovered.....: 4/10 (40.00%) Digests
Progress.....: 62/62 (100.00%)
Rejected.....: 0/62 (0.00%)
Restore.Point....: 1/1 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-62 Iteration:0-62
Candidates.#1....: s → X

The wordlist or mask that you are using is too small.
This means that hashcat cannot use the full parallel power of your device(s).
Unless you supply more work, your cracking speed will drop.
For tips on supplying more work, see: https://hashcat.net/faq/morework

Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Name.....: MD5
Hash.Target.....: hashes_md5.txt
Time.Started.....: Thu Apr  9 06:16:40 2026 (0 secs)
Time.Estimated...: Thu Apr  9 06:16:40 2026 (0 secs)
Guess.Mask.....: ?1?2 [2]
Guess.Charset....: -1 ?l?d?u, -2 ?l?d, -3 ?l?d*!$@_, -4 Undefined
Guess.Queue.....: 2/4 (50.00%)
Speed.#1.....: 2245.1 kH/s (0.03ms) @ Accel:1024 Loops:62 Thr:1 Vec:8
Recovered.....: 4/10 (40.00%) Digests
Progress.....: 2232/2232 (100.00%)
Rejected.....: 0/2232 (0.00%)
Restore.Point....: 36/36 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-62 Iteration:0-62
Candidates.#1....: sa → Xq

The wordlist or mask that you are using is too small.
This means that hashcat cannot use the full parallel power of your device(s).

```

```

Unless you supply more work, your cracking speed will drop.
For tips on supplying more work, see: https://hashcat.net/faq/morework

Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Name.....: MD5
Hash.Target.....: hashes_md5.txt
Time.Started.....: Thu Apr  9 06:16:40 2026 (0 secs)
Time.Estimated...: Thu Apr  9 06:16:40 2026 (0 secs)
Guess.Mask.....: ?1?2?2 [3]
Guess.Charset....: -1 ?l?d?u, -2 ?l?d, -3 ?l?d*!$@_, -4 Undefined
Guess.Queue.....: 3/4 (75.00%)
Speed.#1.....: 87151.1 kH/s (0.43ms) @ Accel:1024 Loops:62 Thr:1 Vec:8
Recovered.....: 4/10 (40.00%) Digests
Progress.....: 80352/80352 (100.00%)
Rejected.....: 0/80352 (0.00%)
Restore.Point....: 1296/1296 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-62 Iteration:0-62
Candidates.#1....: sar → Xqx

Approaching final keyspace - workload adjusted.

Session.....: hashcat
Status.....: Exhausted
Hash.Name.....: MD5
Hash.Target.....: hashes_md5.txt
Time.Started.....: Thu Apr  9 06:16:41 2026 (0 secs)
Time.Estimated...: Thu Apr  9 06:16:41 2026 (0 secs)
Guess.Mask.....: ?1?2?2?2 [4]
Guess.Charset....: -1 ?l?d?u, -2 ?l?d, -3 ?l?d*!$@_, -4 Undefined
Guess.Queue.....: 4/4 (100.00%)
Speed.#1.....: 148.8 MH/s (1.19ms) @ Accel:1024 Loops:62 Thr:1 Vec:8
Recovered.....: 4/10 (40.00%) Digests
Progress.....: 2892672/2892672 (100.00%)
Rejected.....: 0/2892672 (0.00%)
Restore.Point....: 46656/46656 (100.00%)
Restore.Sub.#1...: Salt:0 Amplifier:0-62 Iteration:0-62
Candidates.#1....: s7kx → Xqxv

Started: Thu Apr  9 06:16:40 2026
Stopped: Thu Apr  9 06:16:42 2026

(pnu-kali@kali)~$

```

Observe:

- Speed vs success

Brute force is the most general attack because it tries every possible combination, so it can eventually crack passwords even if they are not in a dictionary. However, it

is much slower than dictionary and hybrid attacks, especially as password length and complexity increase. It becomes impractical against long passwords with mixed symbols.

Task 5: John the Ripper Advanced Testing

Run:

`john --wordlist=/usr/share/wordlists/rockyou.txt --format=raw-md5 hashes_md5.txt`

```
pnu-kali@kali: ~  
File Actions Edit View Help  
(pnu-kali@kali)-[~]  
$ john --wordlist=/usr/share/wordlists/rockyou.txt --format=Raw-MD5 hashes_md5.txt  
Created directory: /home/pnu-kali/.john  
Using default input encoding: UTF-8  
Loaded 10 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])  
Warning: no OpenMP support for this hash type, consider --fork=4  
Press 'q' or Ctrl-C to abort, almost any other key for status  
123456 (?)  
password (?)  
Ahmed123 (?)  
3g 0:00:00:00 DONE (2026-04-09 06:20) 6.521g/s 31181Kp/s 31181Kc/s 222973KC/s fuckyooh21..*7;Vamos!  
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably  
Session completed.
```

`john --incremental hashes_md5.txt`

```
(pnu-kali@kali)-[~]  
$ john --incremental hashes_md5.txt  
Warning: detected hash type "LM", but the string is also recognized as "dynamic-md5($p)"  
Use the "--format=dynamic-md5($p)" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "HAVAL-128-4"  
Use the "--format=HAVAL-128-4" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "MD2"  
Use the "--format=MD2" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "mdc2"  
Use the "--format=mdc2" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "mscash"  
Use the "--format=mscash" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "mscash2"  
Use the "--format=mscash2" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "NT"  
Use the "--format=NT" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "Raw-MD4"  
Use the "--format=Raw-MD4" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "Raw-MD5"  
Use the "--format=Raw-MD5" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "Raw-MD5u"  
Use the "--format=Raw-MD5u" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "Raw-SHA1-AxCrypt"  
Use the "--format=Raw-SHA1-AxCrypt" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "ripemd-128"  
Use the "--format=ripemd-128" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "Snefru-128"  
Use the "--format=Snefru-128" option to force loading these as that type instead  
Warning: detected hash type "LM", but the string is also recognized as "ZipMonster"  
Use the "--format=ZipMonster" option to force loading these as that type instead  
Using default input encoding: UTF-8  
Using default target encoding: CP850  
Loaded 20 password hashes with no different salts (LM [DES 256/256 AVX2])  
Warning: poor OpenMP scalability for this hash type, consider --fork=4  
Will run 4 OpenMP threads  
Press 'q' or Ctrl-C to abort, almost any other key for status  
█
```

`john --wordlist=/usr/share/wordlists/rockyou.txt --rules my_pw_hashes.txt`

```
pnu-kali@kali: ~  
File Actions Edit View Help  
  
(pnu-kali@kali)-[~]  
$ john --wordlist=/usr/share/wordlists/rockyou.txt --format=Raw-MD5 hashes_md5.txt  
Created directory: /home/pnu-kali/.john  
Using default input encoding: UTF-8  
Loaded 10 password hashes with no different salts (Raw-MD5 [MD5 256/256 AVX2 8x3])  
Warning: no OpenMP support for this hash type, consider --fork=4  
Press 'q' or Ctrl-C to abort, almost any other key for status  
123456 (?)  
password (?)  
Ahmed123 (?)  
3g 0:00:00:00 DONE (2026-04-09 06:20) 6.521g/s 31181Kp/s 31181Kc/s 222973KC/s fuckyooh21...*7;Vamos!  
Use the "--show --format=Raw-MD5" options to display all of the cracked passwords reliably  
Session completed.  
  
(pnu-kali@kali)-[~]  
$ john --show --format=Raw-MD5 hashes_md5.txt  
?:password  
?:123456  
?:Ahmed123  
  
3 password hashes cracked, 7 left  
  
(pnu-kali@kali)-[~]  
$ john --wordlist=/usr/share/wordlists/rockyou.txt --format=Raw-SHA1 hashes_sha1.txt  
Using default input encoding: UTF-8  
Loaded 10 password hashes with no different salts (Raw-SHA1 [SHA1 256/256 AVX2 8x])  
Warning: no OpenMP support for this hash type, consider --fork=4  
Press 'q' or Ctrl-C to abort, almost any other key for status  
123456 (?)  
password (?)  
Ahmed123 (?)  
3g 0:00:00:00 DONE (2026-04-09 06:21) 5.084g/s 24310Kp/s 24310Kc/s 173842KC/s xCvBnM,..*7;Vamos!  
Use the "--show --format=Raw-SHA1" options to display all of the cracked passwords reliably  
Session completed.  
  
(pnu-kali@kali)-[~]  
$ john --show --format=Raw-SHA1 hashes_sha1.txt  
?:password  
?:123456  
?:Ahmed123  
  
3 password hashes cracked, 7 left
```

```
pnu-kali@kali: ~  
File Actions Edit View Help  
  
$ john --wordlist=/usr/share/wordlists/rockyou.txt --format=Raw-SHA256 hashes_sha256.txt  
Using default input encoding: UTF-8  
Loaded 10 password hashes with no different salts (Raw-SHA256 [SHA256 256/256 AVX2 8x])  
Warning: poor OpenMP scalability for this hash type, consider --fork=4  
Will run 4 OpenMP threads  
Press 'q' or Ctrl-C to abort, almost any other key for status  
123456 (?)  
password (?)  
Ahmed123 (?)  
3g 0:00:00:00 DONE (2026-04-09 06:21) 5.882g/s 28124Kp/s 28124Kc/s 201495KC/s -sevim-...*7;Vamos!  
Use the "--show --format=Raw-SHA256" options to display all of the cracked passwords reliably  
Session completed.  
  
(pnu-kali@kali)-[~]  
$ john --show --format=Raw-SHA256 hashes_sha256.txt  
  
?:password  
?:123456  
?:Ahmed123
```

Compare with Hashcat:

Factor	Hashcat	John
Speed	Usually faster on GPU-based cracking	Usually slower than Hashcat in GPU-heavy workloads
Accuracy	Very accurate if the correct mode and attack are chosen	Also accurate, but format detection can sometimes need more attention
Ease of use	Powerful but can be more technical	Simple for many standard attacks and easy to start with

Hashcat is generally stronger for high-performance cracking, especially when GPU acceleration is available. John the Ripper is also effective and user-friendly for many common tasks, but Hashcat usually provides better speed and more specialized attack modes.

Task 6: Performance Analysis (Important)

Answer:

1. Which attack cracked most passwords?

The dictionary attack and the rule-based attack cracked the most passwords because several of the tested passwords were common or predictable.

2. Which was fastest?

The **dictionary attack** was the fastest because it only tested known words from the wordlist without generating a huge number of combinations.

3. Which failed completely? Why?

The **mask attack** failed completely for some passwords because the tested masks did not match the real password structure. Mask attacks depend heavily on knowing the correct pattern.

4. Which password type resisted all attacks?

The most random and complex passwords, such as **T9\$kP!2vL@7q** or similarly structured passwords, resisted all simple attacks because they were not dictionary-based and had high entropy.

Task 7: Hash Identification Using AI

Activity:

Give ChatGPT a hash:

5f4dcc3b5aa765d61d8327deb882cf99

Required:

- Identify hash type

The hash 5f4dcc3b5aa765d61d8327deb882cf99 is an **MD5** hash.

- Suggest Hashcat mode

The correct Hashcat mode is **-m 0**.

Task 8: Command Explanation

Activity:

Ask ChatGPT:

Explain this command:

hashcat -m 0 -a 0 hashes.txt rockyou.txt

Required:

- Write ChatGPT explanation

This command tells Hashcat to perform a dictionary attack against hashes stored in hashes.txt using the rockyou.txt wordlist. The option -m 0 selects MD5 hash mode, and -a 0 selects a straight dictionary attack. Hashcat will hash each candidate password from the wordlist and compare it to the target hashes until matches are found.

- Explain how it helped you

This explanation helped me understand what each part of the command does, especially the meaning of -m 0 for MD5 and -a 0 for dictionary attack. It also helped me understand how Hashcat uses a wordlist to test password guesses against stored hashes.

Task 9: AI Wordlist Generation

Activity:

Ask ChatGPT:

Generate password patterns using name Ahmed and year 2000

Example Output:

- Ahmed2000
- Ahmed@123
- Ahmed_2000

Required:

- Write at least 10 generated passwords

Ahmed2000
Ahmed@2000
Ahmed_2000
Ahmed123
Ahmed2000!
Ahmed#2000
Ahmed\$2000
Ahmed2000@
A7med2000
Ahmed_123
Ahmed2000#
Ahmed!123

- Test them in Hashcat

These AI-generated patterns are useful because they reflect common human password habits, such as using names, years, symbols, and small variations. This improves cracking success when the target password is based on personal information.

```
(nouf@kali2)-[~]
$ nano ai_wordlist.txt
```

```
File Actions Edit View Help
GNU nano 8.2
Ahmed2000
Ahmed@2000
Ahmed_2000
Ahmed123
Ahmed2000!
Ahmed#2000
Ahmed$2000
Ahmed2000@
A7med2000
Ahmed_123
```

```
(nouf@kali2)-[~]
$ hashcat -m 0 -a 0 hashes_md5.txt ai_wordlist.txt --potfile-disable
hashcat (v6.2.6) starting

OpenCL API (OpenCL 3.0 PoCL 6.0+debian Linux, None+Asserts, RELOC, LLVM 17.0.6, SLEEF, DISTRO, POCL_DEBUG) - Platform #1 [The pocl project]

* Device #1: cpu-haswell-Intel(R) Core(TM) i5-10210U CPU @ 1.60GHz, 708/1480 MB (256 MB allocatable), 2MCU

Minimum password length supported by kernel: 0
Maximum password length supported by kernel: 256

Hashes: 10 digests; 10 unique digests, 1 unique salts
Bitmaps: 16 bits, 65536 entries, 0x0000ffff mask, 262144 bytes, 5/13 rotates
Rules: 1

Optimizers applied:
* Zero-Byte
* Early-Skip
* Not-Salted
* Not-Iterated
* Single-Salt
* Raw-Hash

ATTENTION! Pure (unoptimized) backend kernels selected.
Pure kernels can crack longer passwords, but drastically reduce performance.
If you want to switch to optimized kernels, append -O to your commandline.
See the above message to find out about the exact limits.

Watchdog: Temperature abort trigger set to 90c

* Device #1: Not enough allocatable device memory for this attack.

Started: Sat Apr 18 12:49:08 2026
Stopped: Sat Apr 18 12:49:46 2026

(nouf@kali2)-[~]
$
```

The command was executed but Hashcat could not complete the attack due to insufficient allocatable device memory on the virtual machine. This is a hardware limitation, not a command error.

.....

Task 10: Understanding Mask Attacks

Activity:

Ask ChatGPT:

When should I use mask attack instead of dictionary attack?

A mask attack should be used when the password structure is partially known but the exact password is unknown. For example, if I know the password starts with a capital letter, contains lowercase letters, and ends with two digits, a mask attack is more efficient than testing a full dictionary. A dictionary attack is better when the password is likely based on common words or leaked password lists.

.....

PART 5: Analysis Questions

Q1:

How did ChatGPT improve your understanding?

ChatGPT improved my understanding by explaining hash types, attack modes, and command syntax in a simple way. It helped me distinguish between dictionary, mask, rule-based, hybrid, and brute-force attacks. It also made it easier to understand why some attacks succeed while others fail

.....

.....

Q2:

Did AI improve cracking success rate? Explain.

Yes, AI improved the cracking success rate by suggesting realistic password patterns and helping identify the most suitable attack strategy. For example, AI-generated guesses based on names, years, and symbols can produce better custom wordlists than using only a standard dictionary. However, AI does not replace proper tools; it mainly improves strategy and wordlist quality.

.....
.....

Q3:

Would you rely on AI in ethical hacking ? Why?

Yes, but only as a support tool. AI is useful for explanations, pattern generation, command clarification, and planning attack strategies in authorized testing environments. However, I would not rely on it alone because practical tools, technical judgment, and ethical boundaries are still essential in ethical hacking.

.....
.....